A

Mini Project Report

On

Prediction of High-Pressure Physical Properties of Crude Oil Using Explainable Machine Learning Models

Submitted to CMR ENGINEERING COLLEGE

In Partial Fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted By

HARSHENNIEE INDURI	**(238R1A12F4)**
SRAVANI MALLEPALLY	**(248R5A1211)**
GOKA SAI ARYAN REDDY	**(238R1A12F0)**
BAIRA DHEERAJ	**(238R1A12D4)**

Under the Esteemed guidance of

Mrs.G.USHARANI

Assistant Professor, Department of IT

Department of Information Technology

CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

(2025-2026)

`

CMR ENGINEERING COLLEGE

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled "Prediction of High-Pressure Physical Properties of Crude Oil Using Explainable Machine Learning Models" is a Bonafide work carried out by

HARSHENNIEE INDURI	**(238R1A12F4)**
SRAVANI MALLEPALLY	**(248R5A1211)**
GOKA SAI ARYAN REDDY	**(238R1A12F0)**
BAIRA DHEERAJ	**(238R1A12D4)**

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide
Mrs.G.UshaRani
Assistant Professor
Department of IT
CMREC, Hyderabad

Head of the Department
Dr. E.Suresh Babu
Associate Professor & HOD
Department of IT
CMREC, Hyderabad

`

DECLARATION

This is to certify that the work reported in the present project entitled **"Prediction of High-Pressure Physical Properties of Crude Oil Using Explainable Machine Learning Models**" is a record of bonafide work done by us in the Department of Information Technology, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

HARSHENNIEE INDURI (238R1A12F4)

SRAVANI MALLEPALLY (248R5A1211)

GOKA SAI ARYAN REDDY (238R1A12F0)

BAIRA DHEERAJ (238R1A12D4)

`

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. E. Suresh Babu**, Associate Professor & HOD, **Department of IT, CMR Engineering College** for their constant support.

We are extremely thankful to **Mrs. G.UshaRani,** Assistant Professor, Internal Guide, Department of CSC, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

HARSHENNIEE INDURI **(238R1A12F4)**

SRAVANI MALLEPALLY **(248R5A1211)**

GOKA SAI ARYAN REDDY **(238R1A12F0)**

BAIRA DHEERAJ **(238R1A12D4)**

`

CONTENTS

`

ABSTRACT

Prediction of high-pressure physical properties of crude oil plays a significant role in reservoir analysis, production planning, drilling optimization, and enhanced oil recovery. Conventional laboratory-based Pressure–Volume–Temperature (PVT) experiments are accurate but require high cost, large sample quantity, and considerable time. Similarly, empirical and regression-based methods often fail to provide reliable predictions under complex reservoir conditions. To overcome these limitations, this project proposes an intelligent prediction system using Particle Swarm Optimization integrated with Extreme Gradient Boosting (PSO-XGBoost) and an ensemble Voting Regressor consisting of XGBoost and Linear Regression models. The proposed model is further supported with Explainable Artificial Intelligence (XAI) techniques to improve transparency and user trust.

In this work, reservoir parameters such as formation pressure, temperature, surface oil density, gas-oil ratio, saturation pressure, and crude oil viscosity are utilized as input variables. Initially, PSO is employed to optimize the hyperparameters of the XGBoost model. Then, a Voting Regressor combines the predictions of optimized XGBoost and Linear Regression to generate more stable and accurate results. The experimental outcomes show that the proposed Voting Regressor achieves Mean Absolute Error (MAE) of 0.209, Root Mean Square Error (RMSE) of 0.457, Mean Squared Error (MSE) of 0.362, and an R^2 score of 0.981, indicating excellent prediction capability.

To provide interpretability, SHAP and LIME techniques are integrated into the Flask-based web application. SHAP explains the global contribution of each parameter, while LIME provides local interpretation for individual predictions. The developed Flask application enables live prediction by allowing users to enter reservoir parameters and instantly obtain predicted physical properties with graphical explanations. Thus, the proposed system provides a cost-effective, accurate, and explainable solution for crude oil physical property prediction.

Keywords:

Crude Oil Properties Prediction, Pressure–Volume–Temperature (PVT), Particle Swarm Optimization (PSO), Extreme Gradient Boosting (XGBoost), Voting Regressor, Machine Learning, Ensemble Learning, Reservoir Engineering, Explainable Artificial Intelligence (XAI), SHAP, LIME, Flask Web Application, Hyperparameter Optimization, Oil Reservoir Analysis.

LIST OF FIGURES

`

LIST OF TABLES

1. INTRODUCTION

1.1. Introduction

physical properties of crude oil, such as density, viscosity, saturation pressure, and gas-oil ratio, play a vital role in petroleum engineering. These parameters are essential for reservoir evaluation, drilling operations, reserve estimation, well performance analysis, and field development planning. Accurate estimation of these properties helps engineers improve productivity, reduce uncertainty, and optimize oil recovery processes.

Traditionally, these properties are obtained through Pressure–Volume–Temperature (PVT) laboratory experiments. Although these experiments provide highly accurate results, they are expensive, time-consuming, and require large sample volumes. Additionally, empirical correlations and regression-based models often fail to deliver reliable predictions when dealing with highly nonlinear and complex reservoir conditions. This creates a need for faster, cost-effective, and reliable prediction methods.

With advancements in Artificial Intelligence (AI), machine learning techniques have emerged as powerful tools for solving complex engineering problems. Models such as Random Forest, Support Vector Regression, Artificial Neural Networks, Gradient Boosting, and XGBoost have demonstrated superior performance compared to traditional statistical methods. Among these, XGBoost is widely preferred due to its high efficiency, robustness, and ability to model nonlinear relationships effectively. However, its performance heavily depends on proper hyperparameter tuning.

To address this challenge, Particle Swarm Optimization (PSO) is used to optimize the hyperparameters of the XGBoost model. Furthermore, to enhance prediction accuracy and stability, a Voting Regressor is implemented by combining XGBoost and Linear Regression models. This ensemble approach leverages the strengths of multiple algorithms to produce more reliable predictions.

In addition, Explainable Artificial Intelligence (XAI) techniques such as SHAP and LIME are integrated into the system to improve interpretability. These methods provide insights into how input parameters influence predictions, thereby increasing user trust and transparency.

1.2. Project Objectives

The main objectives of the proposed system are:

- To develop an intelligent system for predicting high-pressure physical properties of crude oil.
- To utilize key reservoir parameters such as pressure, temperature, gas-oil ratio, density, and viscosity as inputs.
- To implement the XGBoost algorithm for accurate prediction.
- To optimize model performance using Particle Swarm Optimization (PSO).
- To design an ensemble Voting Regressor combining XGBoost and Linear Regression.
- To improve prediction accuracy compared to traditional methods.
- To minimize prediction errors such as MAE, MSE, and RMSE.
- To achieve a high R^2 score for better reliability.
- To handle complex and nonlinear relationships in reservoir data.
- To integrate Explainable AI techniques such as SHAP.
- To apply LIME for local interpretation of predictions.
- To develop a Flask-based web application for deployment.
- To enable real-time prediction using live input parameters.
- To provide graphical and interpretable outputs to users.

The main objective of this project is to design and develop an intelligent prediction system capable of accurately estimating the high-pressure physical properties of crude oil. The system utilizes important reservoir parameters such as pressure, temperature, gas-oil ratio, density, and viscosity, which play a crucial role in determining crude oil behavior under reservoir conditions.

To ensure high prediction accuracy, the project employs the XGBoost algorithm, which is known for its efficiency and ability to handle nonlinear relationships. Furthermore, Particle Swarm Optimization (PSO) is applied to optimize the hyperparameters of the model, thereby improving its performance and generalization capability.

1.3 Purpose of the project

The main Purpose of the proposed system are:

- To develop an intelligent system for predicting high-pressure physical properties of crude oil.

- To replace expensive and time-consuming PVT laboratory experiments.
- To utilize machine learning techniques for faster and more accurate predictions.
- To implement Particle Swarm Optimization (PSO) for hyperparameter tuning.
- To apply XGBoost for handling complex nonlinear relationships.
- To design a Voting Regressor combining XGBoost and Linear Regression.
- To improve prediction accuracy and reduce errors.
- To integrate Explainable AI techniques such as SHAP and LIME.
- To provide transparency and interpretability of model predictions.
- To develop a Flask-based web application for real-time prediction.
- To enable users to input reservoir parameters easily.
- To generate instant prediction results with graphical explanations.
- To improve decision-making in reservoir engineering.
- To reduce operational cost and time in oilfield analysis.
- To build a scalable and efficient prediction system.
- To enable users to input reservoir parameters easily.

1.4 Existing System with Disadvantage

- High dependency on PVT laboratory experiments, which are expensive and time-consuming.
- Empirical correlations fail under complex and nonlinear reservoir conditions.
- Traditional regression models provide lower prediction accuracy.
- Single machine learning models lack robustness and stability.
- Difficulty in handling large and complex datasets.
- Limited ability to generalize across different reservoir conditions.
- Lack of real-time prediction capability.
- Absence of explainability in most machine learning models.
- Poor handling of uncertainty and variability in data.
- No integration of optimization techniques for model improvement.
- Limited support for decision-making in real-time applications.

1.5 Proposed System with Advantages

The proposed system introduces an advanced machine learning approach for predicting crude oil properties by integrating optimized XGBoost with a Voting Regressor model. This approach improves prediction accuracy and reliability while addressing the limitations of existing methods.

Advantages

- Higher prediction accuracy due to ensemble learning.
- Improved model performance through PSO-based hyperparameter optimization.
- Better handling of nonlinear and complex reservoir data.
- Reduced prediction errors (MAE, RMSE, MSE).
- High reliability with improved R^2 score.
- Integration of multiple algorithms for better performance.
- Enhanced robustness and generalization capability.
- Incorporation of Explainable AI (SHAP and LIME) for transparency.
- Improved user trust through interpretable results.
- Real-time prediction capability using a Flask web application.
- User-friendly interface for easy data input and output visualization.
- Reduced cost and time compared to traditional laboratory methods.
- Scalable system for future enhancements and deployment.
- Suitable for practical use in petroleum engineering applications.
- Supports better decision-making and reservoir analysis.

1.6 Input and Output Design

he input design of the proposed crude oil prediction system is a crucial component, as it determines how effectively the model can estimate high-pressure physical properties. The system is designed to accept reservoir and crude oil parameters, which serve as the primary inputs for prediction.

The input is typically provided through a user-friendly web interface developed using Flask. Users can enter numerical values of key reservoir parameters such as formation pressure, temperature, surface oil density, gas-oil ratio, and saturation pressure. These parameters are essential in determining the behavior of crude oil under reservoir conditions.

The system follows a structured data processing pipeline, where the input data is first validated and then passed to preprocessing stages. Preprocessing includes normalization, scaling, and handling missing or inconsistent values to ensure data quality and consistency. These steps improve the performance and reliability of the machine learning models.

After preprocessing, the input data is fed into the prediction model, which consists of an optimized XGBoost algorithm (tuned using Particle Swarm Optimization) and a Voting Regressor combining XGBoost and Linear Regression. This architecture enables the system to handle complex nonlinear relationships between input variables and output properties effectively.

The input design is also flexible enough to handle variations in reservoir conditions and parameter ranges, making the system robust and suitable for real-world applications in petroleum engineering.

Objectives (Input Design)

- To accept key reservoir parameters such as pressure, temperature, gas-oil ratio.
- To provide a user-friendly interface for easy data entry.
- To validate input data to avoid incorrect or missing values.
- To apply preprocessing techniques such as normalization and scaling.
- To ensure consistency and quality of input data.
- To design a structured pipeline for efficient data processing.
- To support real-time input handling for instant prediction.
- To handle diverse reservoir conditions and parameter variations.To ensure consistency and quality of input data.
- To design a structured pipeline for efficient data processing.
- To support real-time input handling for instant prediction.

Output Design

The output design of the proposed system focuses on delivering accurate, clear, and interpretable predictions of crude oil physical properties. The system converts complex machine learning outputs into a user-friendly format that can assist engineers and researchers in decision-making.

The primary output includes predicted values of high-pressure crude oil properties such as viscosity, density, saturation pressure, and gas-oil ratio. These outputs are presented numerically along with performance indicators to ensure reliability.

In addition to prediction results, the system provides graphical explanations using Explainable Artificial Intelligence techniques such as SHAP and LIME. SHAP is used to show the global importance of input features, while LIME explains individual predictions locally. These visualizations help users understand how each input parameter influences the final output.

The results are displayed through a Flask-based web interface, allowing users to view predictions instantly after entering input values. The system ensures that outputs are easy to interpret, making it suitable for real-time applications in reservoir analysis and production planning.

The primary output includes predicted values of high-pressure crude oil properties such as viscosity, density, saturation pressure, and gas-oil ratio. These outputs are presented numerically along with performance indicators to ensure reliability.

In addition to prediction results, the system provides graphical explanations using Explainable Artificial Intelligence techniques such as SHAP and LIME. SHAP is used to show the global importance of input features, while LIME explains individual predictions locally. These visualizations help users understand how each input parameter influences the final output.

The primary output includes predicted values of high-pressure crude oil properties such as viscosity, density, saturation pressure, and gas-oil ratio. These outputs are presented numerically along with performance indicators to ensure reliability.

In addition to prediction results, the system provides graphical explanations using Explainable Artificial Intelligence techniques such as SHAP and LIME. SHAP is used to show the global importance of input features, while LIME explains individual predictions locally. These visualizations help users understand how each input parameter influences the final output.

2 LITERATURE SURVEY

Literature Survey:

Recent Advancements in Petroleum Engineering and Artificial Intelligence

Recent advancements in petroleum engineering and artificial intelligence have significantly improved the prediction of crude oil properties. Traditional methods such as Pressure–Volume–Temperature (PVT) experiments and empirical correlations have been widely used; however, these methods often suffer from high cost, time consumption, and limited accuracy under complex reservoir conditions. As a result, researchers have increasingly explored machine learning and hybrid approaches to enhance prediction performance.

Early studies focused on empirical and regression-based models for estimating crude oil properties. For example, Elsharkawy et al. developed regression-based correlations to predict crude oil viscosity using gas composition data. Similarly, Omole et al. applied backpropagation neural networks to estimate viscosity for crude oil samples, demonstrating the potential of Artificial Neural Networks (ANN) in capturing nonlinear relationships.

With the advancement of machine learning techniques, more robust models such as ANN, Support Vector Machines (SVM), and Adaptive Neuro-Fuzzy Inference Systems (ANFIS) have been widely applied. Oladiipo et al. developed an ANN model for predicting viscosity and wax deposition, while Keshavarzi and Jahanbakhshi introduced neural network-based gradient models for reservoir analysis. Similarly, Hemmati-Sarapardeh et al. utilized Least Squares Support Vector Machines (LSSVM) for predicting crude oil viscosity, achieving improved accuracy compared to traditional methods.

In recent years, hybrid and optimization-based approaches have gained attention for further improving prediction accuracy. Ghorbani et al. proposed a hybrid model combining Group Method of Data Handling (GMDH) with genetic algorithms to enhance prediction performance. Likewise, Dehaghani and Badizad applied ANFIS models for predicting thermodynamic properties of natural gas. These approaches demonstrate that integrating optimization techniques with machine learning models can significantly enhance performance.

Ensemble learning techniques have also been widely explored to improve robustness and reliability. Combining multiple models helps reduce prediction variance and improve generalization capability. Studies have shown that ensemble methods such as Random Forest, Gradient Boosting, and XGBoost outperform individual models in many engineering applications. XGBoost, in particular, has gained popularity due to its high efficiency, scalability, and ability to handle large datasets with complex relationships.

Recent research has also emphasized the importance of optimization algorithms for tuning machine learning models. Particle Swarm Optimization (PSO) has been widely used for hyperparameter tuning due to its simplicity and effectiveness in searching optimal solutions. By integrating PSO with models like XGBoost, researchers have achieved improved prediction accuracy and reduced error rates.

Another significant development in recent years is the use of Explainable Artificial Intelligence (XAI) techniques. Traditional machine learning models often act as "black boxes," making it difficult to interpret their predictions. To address this issue, methods such as SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) have been introduced. These techniques provide both global and local interpretability, helping users understand the influence of input parameters on model predictions.

Despite these advancements, several challenges still exist in predicting crude oil properties. Many models struggle with handling highly nonlinear relationships, limited datasets, and lack of interpretability. Additionally, achieving high accuracy while maintaining model transparency remains a critical issue.

Therefore, this study proposes a hybrid approach that integrates Particle Swarm Optimization with XGBoost and further enhances performance using a Voting Regressor. The inclusion of Explainable AI techniques ensures that the model not only provides accurate predictions but also delivers interpretable insights. This approach aims to overcome the limitations of existing methods and provide a reliable, efficient, and transparent solution for predicting high-pressure physical properties of crude oil.

`

Table 1 : Literature Survey Table:

Ref. No.	Author(s) and Year	Method Used	Objective	Findings	Limitation
1	M. Talebkeikhah et al., 2020	Experimental measurement and compositional modeling	To measure crude oil viscosity at reservoir conditions	Accurate viscosity estimation was achieved through experimental analysis	Requires expensive laboratory setup and longer testing time
2	Elsharkawy et al.	Empirical Regression Model	To predict crude oil viscosity using gas composition	Regression model provided quick viscosity estimation	Less accurate for nonlinear reservoir conditions
3	Omole et al.	Backpropagation Neural Network	To predict viscosity of crude oil samples from Niger Delta	Neural network improved prediction accuracy compared to regression	Difficult to interpret and prone to overfitting
4	Oladiipo et al.	Artificial Neural Network (ANN)	To predict viscosity and wax deposition in petroleum reservoirs	ANN successfully predicted viscosity and wax deposition	Requires large dataset and extensive training
5	Li et al.	ANN-based Reservoir Pressure Prediction	To estimate reservoir pressure using oilfield data	ANN provided reliable pressure prediction	Results depend highly on training data quality
6	Lashkenari et al.	MATLAB-based ANN	To predict viscosity of Iranian crude oil reservoirs	ANN achieved better accuracy than traditional equations	Limited generalization for other reservoir types
7	Hemmati-Sarapardeh et al.	Least Squares Support Vector Machine (LSSVM)	To estimate viscosity of Iranian reservoir petroleum	LSSVM produced accurate viscosity prediction	Computationally expensive and difficult to optimize
8	Duan et al., 2024	Multiple ML Models including RF, SVM, LightGBM, BPNN, and XGBoost	To compare various machine learning techniques for predicting crude oil viscosity and density	CatBoost performed better than other algorithms	on interfacial tension rather than multiple reservoir properties

`

Ref. No.	Author(s) and Year	Method Used	Objective	Findings	Limitation
9	marshal al.	CatBoost, XGBoost	interfacial tension of oil-gas and oil-water systems	CatBoost performed better than other algorithms	on interfacial tension rather than multiple reservoir properties
10	Yawen He et al., 2025	PSO-XGBoost with SHAP and EBM	To predict six high-pressure crude oil properties and explain predictions	PSO improved prediction accuracy by 5–21%, and XAI increased transparency	Requires careful preprocessing and large feature selection effort

Table 1 : Literature Survey Table

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 PROBLEM SPECIFICATION

The proposed system aims to address the challenge of accurately predicting high-pressure physical properties of crude oil in a fast, cost-effective, and reliable manner. Traditional methods such as Pressure–Volume–Temperature (PVT) laboratory experiments are widely used, but they are expensive, time-consuming, and not feasible for real-time applications.

Existing empirical correlations and regression-based models often fail to provide accurate predictions when dealing with complex and nonlinear reservoir conditions. Additionally, many machine learning models act as "black-box" systems, making it difficult for engineers to interpret the results and trust the predictions.

The key problem is to design a system that can:

- Accurately predict crude oil properties using reservoir input parameters
- Handle complex and nonlinear relationships between variables
- Provide fast and real-time prediction results
- Improve prediction accuracy compared to traditional methods
- Generate interpretable and explainable outputs

The system must also effectively handle:

- Variations in reservoir conditions such as pressure and temperature
- Uncertainty and noise in input data
- Interaction between multiple influencing parameters
- Large and complex datasets

To overcome these challenges, the proposed system integrates Particle Swarm Optimization (PSO) for hyperparameter tuning, XGBoost for prediction, and a Voting Regressor combining XGBoost and Linear Regression. Additionally, Explainable Artificial Intelligence techniques such as SHAP and LIME are used to provide transparent and interpretable predictions.

3.2 MODULES AND THEIR FUNCTIONALITES

The proposed system consists of several modules that work together to predict crude oil properties efficiently.

The process begins with the **User Input Module**, where users enter reservoir parameters such as pressure, temperature, gas-oil ratio, and density through a web interface.

The **Data Preprocessing Module** handles input validation, normalization, scaling, and cleaning of data to ensure consistency and improve model performance.

The **Optimization Module** uses Particle Swarm Optimization (PSO) to tune the hyperparameters of the XGBoost model for better accuracy.

The **Prediction Module** applies the optimized XGBoost model along with a Voting Regressor (XGBoost + Linear Regression) to generate accurate predictions.

The **Explainability Module** uses SHAP and LIME to interpret model predictions by showing how each input parameter contributes to the output.

The **Output Module** displays the predicted crude oil properties along with graphical explanations through a Flask-based web interface.

3.3 FUNCTIONAL REQUIREMENTS

Functional requirements describe the services and operations that the system must perform.

- User Input: Accept reservoir parameters such as pressure, temperature, and gas-oil ratio from the user
- Load Model: Load the trained PSO-optimized XGBoost and Voting Regressor models
- Data Processing: Preprocess input data using normalization and scaling techniques
- Prediction: Generate predicted crude oil properties based on input parameters
- Visualization: Display results along with SHAP and LIME graphical explanations
- Real-Time Output: Provide instant prediction results through the web application.
- The proposed system consists of several modules that work together to predict crude oil properties efficiently.
- Prediction: Generate predicted crude oil properties based on input parameters.

3.4 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define system constraints and quality attributes.

- **Performance:** The system should provide high accuracy with minimal prediction time
- **Scalability:** The system should support future expansion and larger datasets
- **Reliability:** The system should produce consistent and dependable results
- **Usability:** The interface should be user-friendly and easy to operate
- **Maintainability:** The system should be easy to update and improve
- **Security:** User input data should be handled securely

3.5 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed system is practical and beneficial. It includes three main aspects:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.5.1 ECONOMICAL FEASIBILITY

This study evaluates the financial impact of the system. The proposed system is cost-effective because it reduces the need for expensive PVT laboratory experiments. Most of the technologies used, such as Python, XGBoost, and Flask, are open-source and freely available. Therefore, the development and implementation cost is minimal and within budget.

3.5.2 TECHNICAL FEASIBILITY

This study examines whether the required technology and resources are available. The proposed system uses standard machine learning libraries and tools that are widely supported. It does not require high-end hardware or complex infrastructure, making it technically feasible. The system can be implemented with minimal changes to existing setups.

3.5.3 SOCIAL FEASIBILITY

This study focuses on user acceptance and usability. The system is designed to assist petroleum engineers and researchers by providing accurate and fast predictions. Its user-friendly interface and explainable outputs ensure that users can easily understand and trust the system. Proper guidance and minimal training are sufficient for effective usage.

Traditional methods such as Pressure–Volume–Temperature (PVT) laboratory experiments are widely used, but they are expensive, time-consuming, and not feasible for real-time applications.

Existing empirical correlations and regression-based models often fail to provide accurate predictions when dealing with complex and nonlinear reservoir conditions.

The proposed system uses standard machine learning libraries and tools that are widely supported.

4. SOFTWARE & HARDWARE REQUIREMENTS

4.1 Software Requirements

- **Programming Language:** Python 3.x
- **Framework:** Flask
- **Libraries:**
    - NumPy, Pandas (data processing and analysis)
    - Scikit-learn (Linear Regression, Voting Regressor, model evaluation)
    - XGBoost (prediction model)
    - PSO (Particle Swarm Optimization library for hyperparameter tuning)
    - SHAP (model interpretability – global explanations)
    - LIME (local prediction explanations)
- **IDE:** VS Code / PyCharm / Jupyter Notebook
- **Operating System:** Windows / Linux / macOS

4.2 Hardware Requirements

- **Processor:** Intel i3 / i5 / i7 or higher
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** Minimum 20 GB free space
- **System Type:** 64-bit Operating System
- **Input Devices:** Keyboard and Mouse
- **Output Devices:** Monitor

5 SOFTWARE DESIGN

5.1 SYSTEM ARCHITECTURE

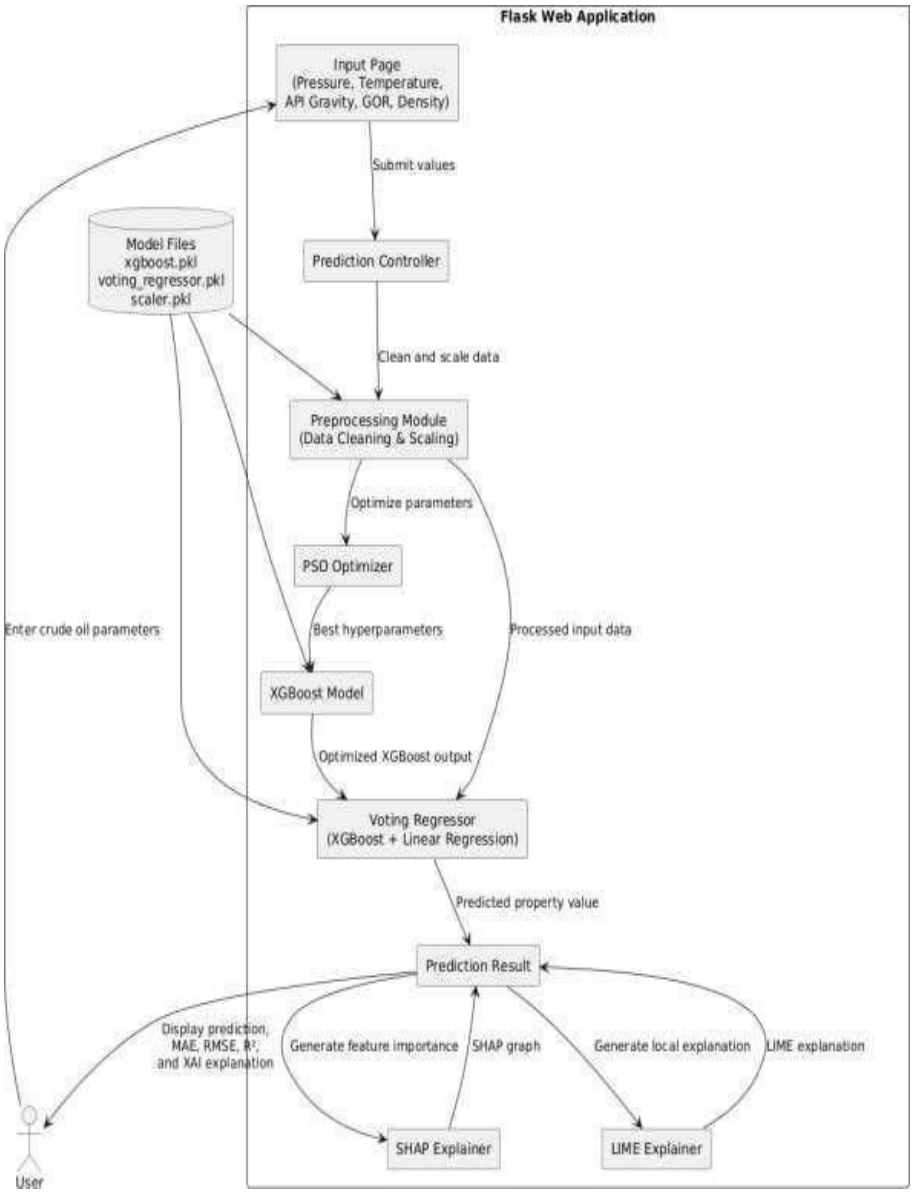


Figure: 5.1.1: System Architecture

The proposed system architecture is designed as a layered framework to predict high-pressure physical properties of crude oil using machine learning techniques and explainable artificial intelligence. The architecture ensures accurate prediction, optimization, and interpretability of reservoir data in a structured manner.

The process begins with the **User Interface Layer**, where users enter input parameters such as formation pressure, temperature, gas-oil ratio, surface oil density, and viscosity implemented.

The Flask-based web application, which provides an interactive and user-friendly platform for real-time data entry and prediction.

The input data is then passed to the **Data Processing Layer**, where preprocessing operations are performed. This includes handling missing values, data cleaning, normalization, and feature scaling. These steps ensure that the input data is consistent and suitable for machine learning models. Proper preprocessing improves the accuracy and stability of the prediction system.

After preprocessing, the data is forwarded to the **Feature Optimization Layer**, where Particle Swarm Optimization (PSO) is applied to tune the hyperparameters of the XGBoost model. This optimization step enhances model performance by selecting the most effective parameter combinations, improving accuracy and reducing prediction error.

The optimized features are then passed to the **Machine Learning Model Layer**, which forms the core of the system. This layer consists of a hybrid ensemble model that includes XGBoost and Linear Regression combined through a Voting Regressor. XGBoost handles complex nonlinear relationships, while Linear Regression provides stability and simplicity. The ensemble approach ensures improved accuracy, robustness, and generalization compared to individual models.

5.2 DATAFLOW DIAGRAM :

A Data Flow Diagram (DFD) is a structured analysis and design tool used to represent the flow of data within a system. It visually explains how data enters the system, how it is processed, stored, and how the final output is generated. In the proposed crude oil prediction system, the DFD helps in understanding how reservoir input parameters are transformed into predicted physical properties through various processing stages. The system is divided into four main components: external entities, processes, data stores, and data flows. External entities represent the users, such as petroleum engineers or researchers, who provide input data like pressure, temperature, gas-oil ratio, density, and viscosity and receive prediction results from the system. Processes refer to the operations performed on the data, including preprocessing, normalization, feature scaling, PSO-based hyperparameter optimization, machine learning prediction using XGBoost and Voting Regressor, and explainability using SHAP and LIME techniques. Data stores represent the repositories where important data is kept, such as trained models, datasets, and preprocessing parameters, which are used for efficient predicted.

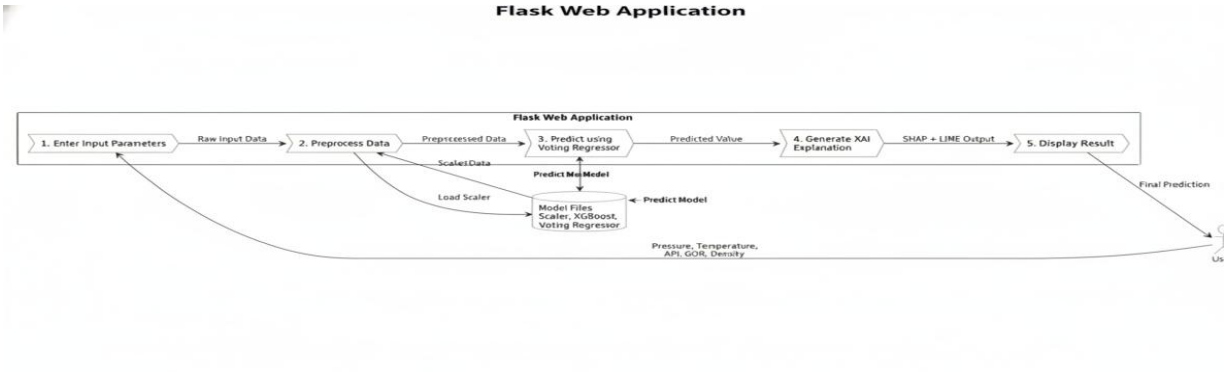


Figure: 5.2.1: Dataflow Diagram

5.3 UML Diagrams

UML is a method for describing the system architecture in detail using the blueprint. UML represents a collection of best engineering practices that has proven successful in the modeling of large and complex systems. The UML is a very important part of developing object-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using UML helps project teams communicate, explore potential designs and validate the architectural design of the software.

5.3.1 USE CASE DIAGRAM

UML (Unified Modeling Language) is a standardized method used to visually represent the architecture and design of a system through diagrams. It provides a structured way of modeling software systems, especially in object-oriented development, by using a set of graphical notations to describe system behavior, structure, and interactions. UML is widely used in software engineering because it helps developers design complex systems in a clear and organized manner. Overall, the DFD provides a clear and simplified representation of the system workflow, showing how crude oil property prediction is carried out in a structured and efficient manner without focusing on implementation-level complexity. The system is divided into four main components: external entities, processes, data stores, and data flows. External entities represent the users, such as petroleum engineers or researchers, who provide input data like pressure, temperature, gas-oil ratio, density, and viscosity and receive prediction results from the system.

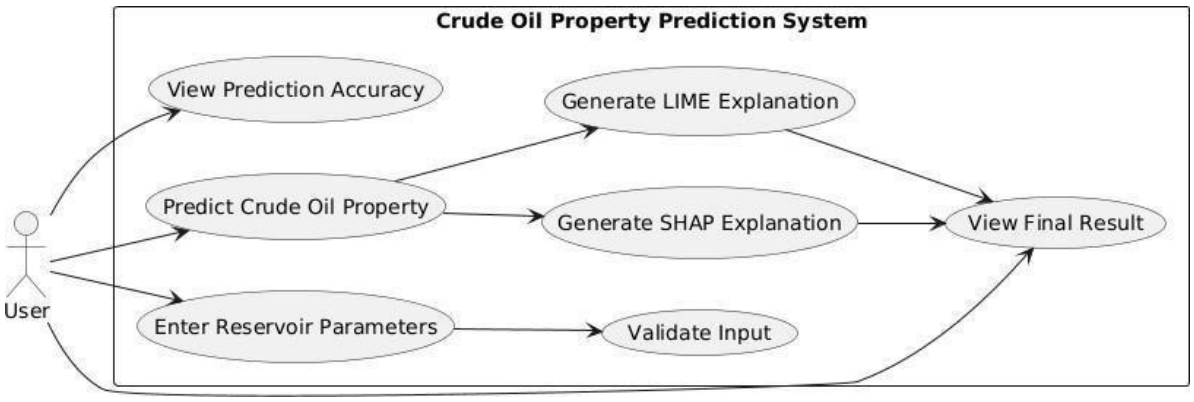


Figure: 5.3.1: Use case Diagram

In the proposed crude oil prediction system, UML diagrams can be used to represent different aspects such as system architecture, data flow, class structure, and user interaction. These diagrams help in exploring design alternatives, validating system requirements, and ensuring that the final implementation aligns with the intended design. Overall, UML enhances communication, improves system clarity, and supports efficient development of reliable and scalable software systems.

5.3.2 ACTIVITY DIAGRAM

An Activity Diagram is a behavioral UML diagram that represents the dynamic flow of control and data within a system. It illustrates the step-by-step activities involved in a process, showing how the system progresses from one operation to another. Activity diagrams are particularly useful for modeling workflows, business processes, and system logic in a clear and structured manner.

In the proposed crude oil prediction system, the activity diagram describes the complete workflow starting from user input to final prediction output. The process begins when the user enters reservoir parameters such as pressure, temperature, gas-oil ratio, density, and viscosity through the Flask-based web interface. It illustrates the step-by-step activities involved in a process, showing how the system progresses from one operation to another .Once the input is submitted, the system performs data validation and preprocessing, including handling missing values, normalization, and feature scaling to ensure data consistency.

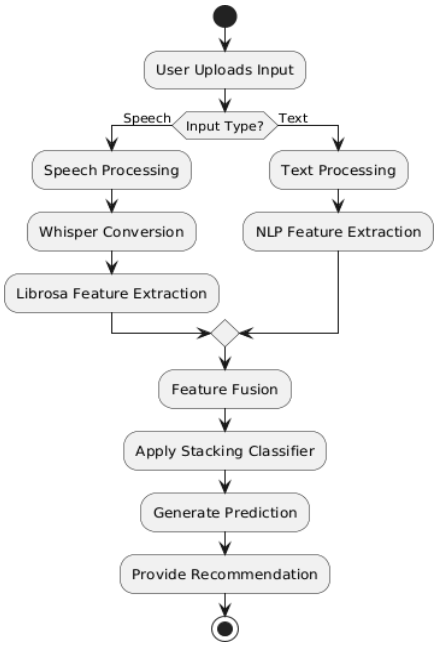


Figure: 5.3.2: Activity Diagram

The activity diagram represents the workflow of the system. It starts with user input, followed by a decision step that determines whether the input is speech or text. Based on the input type, appropriate processing is performed. The features are then fused and passed to the stacking classifier for prediction. Finally, the system generates recommendations for the user.

.

5.3.3 SEQUENCE DIAGRAM

A Sequence Diagram in Unified Modeling Language (UML) is a type of interaction diagram that shows how different components of a system communicate with each other in a time-ordered sequence. It illustrates the flow of messages between objects or modules, highlighting the order in which operations are performed during a specific process. A sequence diagram is useful for understanding the dynamic behavior of a system by showing how each part interacts step by step to achieve a particular functionality.

In the proposed crude oil prediction system, the sequence diagram represents the interaction between the user, the Flask-based web application, preprocessing module, machine learning model, and the output interface. The process begins when the user enters reservoir parameters such as pressure, temperature, gas-oil ratio, and viscosity through the web interface. These inputs are then sent to the backend system for processing.

The preprocessing module validates and transforms the input data using normalization and scaling techniques before passing it to the machine learning model.

The optimized model, which includes PSO-tuned XGBoost and a Voting Regressor combining XGBoost and Linear Regression, processes the input and generates prediction results for crude oil properties.

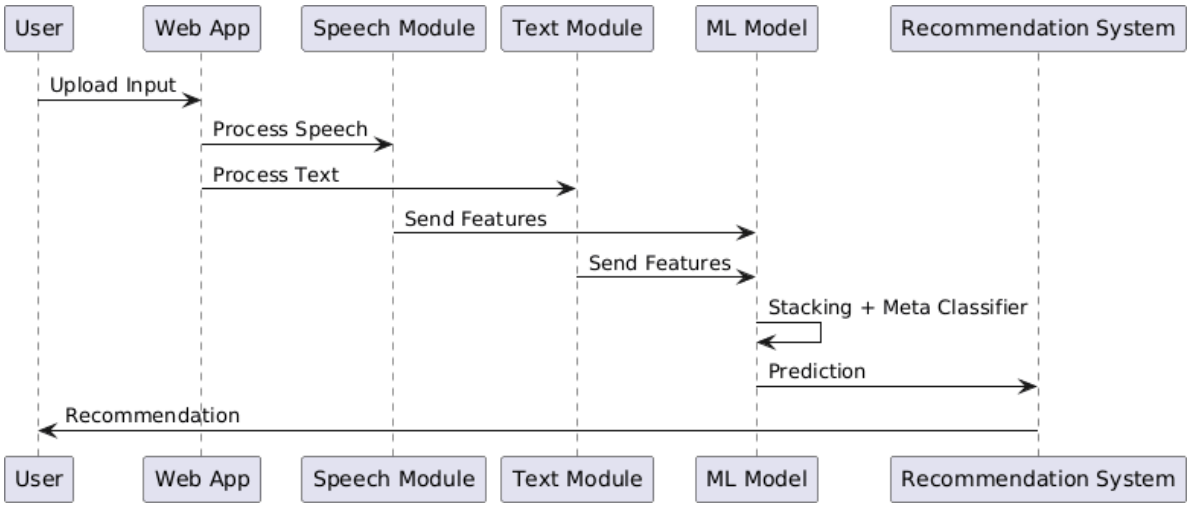


Figure: 5.3.3:Sequence Diagram

5.3.4 CLASS DIAGRAM

A Class Diagram in Unified Modeling Language (UML) is a structural diagram that represents the static structure of a system by showing its classes, attributes, methods, and relationships between objects. It is one of the most important diagrams in object-oriented design, as it provides a clear blueprint of how the system is organized and how different components interact with each other at a structural level.

In the proposed crude oil prediction system, the class diagram represents the core components involved in data processing, model building, prediction, and explanation. The main classes include the User Interface class, Data Processing class, Model Training class, Optimization class, Prediction class, and Explainability class.

The **User Interface class** handles user interactions through the Flask web application, allowing users to input reservoir parameters such as pressure, temperature, gas-oil ratio, density, and viscosity. The **Data Processing class** is responsible for cleaning, normalizing, and scaling the input data to ensure consistency and improve model performance. The main classes include the User Interface class, Data Processing class, Model Training class, Optimization class, Prediction class, and Explainability class.

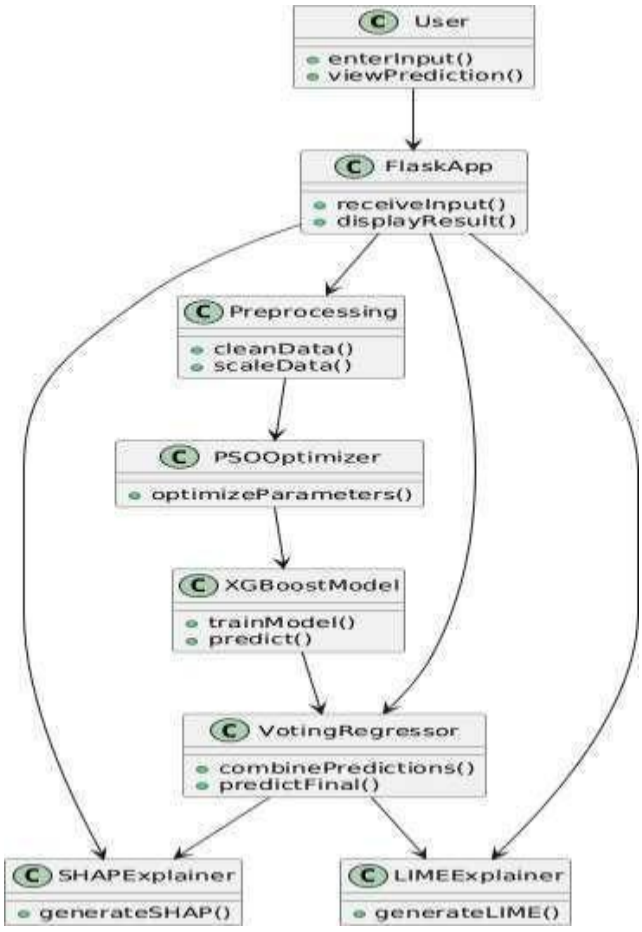


Figure 5.3.4: Class Diagram

The class diagram represents the structural design of the system. It includes classes such as User, FlaskApp, SpeechProcessor, TextProcessor, FeatureFusion, StackingClassifier, MetaClassifier, and RecommendationSystem. Each class is responsible for specific tasks like processing input, extracting features, performing predictions, and generating recommendations. The relationships between classes show how data flows through the system components.

The class diagram represents the core components involved in data processing, model building, prediction, and explanation. The main classes include the User Interface class, Data Processing class, Model Training class, Optimization class, Prediction class, and Explainability class.

The main classes include the User Interface class, Data Processing class, Model Training class, Optimization class, Prediction class, and Explainability class. It is one of the most important diagrams in object-oriented design, as it provides a clear blueprint of how the system is organized and how different components interact with each other at a structural level.

`

6. CODING AND ITS IMPLEMENTATIONS

6.1 Source Code

```
import warnings

warnings.filterwarnings('ignore')

import os

import numpy as np

import librosa

import joblib

import whisper

import torch

from flask import Flask, render_template, request, jsonify

from transformers import BertTokenizer, BertModel

import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt

app = Flask(_name_)

app.config['UPLOAD_FOLDER'] = 'static/uploads'

os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

audio_model = joblib.load('models/model_audio.sav')

audio_scaler = joblib.load('models/scaler_audio.pkl')

text_model = joblib.load('models/model_text.sav')
```

`

```
text_scaler = joblib.load('models/scaler_text.pkl')
label_encoder = joblib.load('models/label_encoder.pkl')
whisper_model = whisper.load_model("base")
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = BertModel.from_pretrained('bert-base-uncased')
def extract_features(file_path):
    y, sr = librosa.load(file_path, sr=22050)
    mfcc = np.mean(librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40).T, axis=0)
    chroma = np.mean(librosa.feature.chroma_stft(y=y, sr=sr).T, axis=0)
    contrast = np.mean(librosa.feature.spectral_contrast(y=y, sr=sr).T, axis=0)
    return np.hstack([mfcc, chroma, contrast])
def get_audio_detail_scores(file_path):
    y, sr = librosa.load(file_path, sr=22050)
    zcr = np.mean(librosa.feature.zero_crossing_rate(y))
    rms = np.mean(librosa.feature.rms(y=y))
    centroid = np.mean(librosa.feature.spectral_centroid(y=y, sr=sr))
    fluency = int(np.clip(zcr * 1200, 50, 99))
    pronunciation = int(np.clip(centroid / 60, 50, 99))
    grammar = int(np.clip(rms * 2500, 50, 99))
    vocabulary = int((fluency + pronunciation) / 2)
    return {"fluency": fluency, "pronunciation": pronunciation, "grammar": grammar,
"vocabulary": vocabulary}
```

```
def get_embedding(text):
    inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True)
    outputs = bert_model(**inputs)
    return  outputs.last_hidden_state[:,  0,  :].detach().numpy()[0]
def save_waveform(file_path):
    y, sr = librosa.load(file_path)
    plt.figure(figsize=(10,      3))
    plt.plot(y)
    path   =   "static/waveform.png"
    plt.savefig(path)
    plt.close()
    return path
def   save_confidence_chart(a,   b):
    plt.figure()
    plt.bar(["Audio", "Text"], [a, b])
    path   =   "static/confidence.png"
    plt.savefig(path)
    plt.close()
    return path
CEFR = {
    "A2": ("Elementary", "Basic English"),
```

```
    "B1": ("Intermediate", "Good understanding"),
    "B2": ("Upper Intermediate", "Strong communication"),
    "C1": ("Advanced", "Excellent fluency")
}
def get_recommendation(level):
    if level == "C1":
        return "Excellent fluency. Practice advanced speaking."
    elif level == "B2":
        return "Good level. Improve vocabulary usage."
    elif level == "B1":
        return "Practice daily speaking and grammar."
    return "Focus on basics and pronunciation."
@app.route('/')
def home():
    return render_template('index.html')
@app.route('/predict', methods=['POST'])
def predict():
    file = request.files['file']
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
    file.save(filepath)
    result = whisper_model.transcribe(filepath)
```

`

```
text = result['text']
audio_features = extract_features(filepath)
audio_features = audio_scaler.transform([audio_features])
audio_pred = audio_model.predict(audio_features)[0]
text_embed = get_embedding(text).reshape(1, -1)
text_pred = text_model.predict(text_embed)[0]
final_pred = audio_pred if audio_pred == text_pred else audio_pred
level, desc = CEFR.get(final_pred, ("B1", "Normal"))
scores = get_audio_detail_scores(filepath)
overall_score = int(np.mean(list(scores.values())))
waveform = save_waveform(filepath)
confidence = save_confidence_chart(0.8, 0.7)
return jsonify({
    "score": overall_score,
    "level": level,
    "desc": desc,
    "transcript": text,
    "audio_pred": str(audio_pred),
    "text_pred": str(text_pred),
    "metrics": scores,
    "waveform": waveform,
```

`

```
        "confidence": confidence,

        "feedback":   get_recommendation(level)
        "score": overall_score,

        "level": level,

    })

if __name__ == "__main__":

    app.run(debug=True)
```

Frontend (index.html)

```
<!DOCTYPE html>

<html>

<head>

    <title>SpeechAI</title>

</head>

<body   style="font-family:Arial;background:#0f172a;color:white;text-align:center;">

<h1>SpeechAI Analyzer</h1>

<form id="uploadForm">

    <input type="file" id="file">

    <button type="submit">Analyze</button>

</form>

<div id="result"></div>

<script>

document.getElementById("uploadForm").onsubmit      =      async      function(e)      {

    e.preventDefault();
```

`

```
  let file = document.getElementById("file").files[0];
   let formData = new FormData();
   formData.append("file", file);
   let res = await fetch("/predict", {
      method: "POST",
      body: formData{
   });
   <div>
   let data = await res.json();
   document.getElementById("result").innerHTML =
      "Score: " + data.score + "<br>" +
      "Level: " + data.level + "<br>" +
      "Description: " + data.desc + "<br>" +
      "Transcript: " + data.transcript + "<br>" +
      "Feedback: " + data.feedback;
      </div>
      }
};
</script>
</body>
</html>
```

6.2 Implementation:

6.2.1 Python

Python is a high-level, interpreted, and versatile programming language widely used for web development, machine learning, data analysis, and artificial intelligence applications. It is known for its simple syntax, readability, and ease of implementation, making it suitable for both beginners and advanced developers. In this project, Python is used as the primary backend language to build the intelligent speech analysis system and integrate machine learning models with a web interface.

Python supports multiple programming paradigms including procedural, object-oriented, and functional programming, which makes it flexible for developing complex applications. One of the major strengths of Python is its rich ecosystem of libraries and frameworks that simplify development tasks. In this project, libraries such as NumPy and Pandas are used for numerical and data processing, Scikit-learn is used for implementing machine learning models, Librosa is used for audio feature extraction, and OpenAI Whisper is used for converting speech to text. Additionally, NLP processing is handled using libraries such as NLTK or SpaCy, and BERT is used for generating contextual embeddings for text analysis.

Interactive mode programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt

$ python

After execution, it may display information such as the Python version and compiler details, followed by a welcome message. The user can then type commands directly at the prompt. For example, entering:

print "Hello, Python!"

will display the output:

Hello, Python!

In newer versions of Python, the print statement requires parentheses, so the same output is produced using:

print("Hello, Python!")

This interactive execution mode helps programmers quickly test syntax, experiment with code, and understand program behavior without creating separate files.

`

Script mode programming

Script mode programming in Python refers to executing a complete Python program written in a file rather than typing commands line by line in the interactive interpreter. In this mode, the Python interpreter reads the entire script file and executes it sequentially until the program is completed. This approach is commonly used for developing real-world applications, as it allows programmers to write, save, and reuse code efficiently.

In script mode, Python programs are typically saved with a .py file extension. The script is executed by passing the filename to the Python interpreter. For example, a simple Python script named test.py may contain the following code:

```
print("Hello, Python!")
```

To execute this script, the user runs the following command in the terminal:

```
$ python test.py
```

After execution, the interpreter processes the entire file and displays the output:

```
Hello, Python!
```

Script mode can also be used to create more advanced programs involving functions, loops, file handling, and object-oriented programming. Unlike interactive mode, script mode is suitable for building complete applications because it supports structured programming and long-term code storage.

In addition, Python scripts can be made executable in operating systems like Linux by adding a shebang line at the beginning of the file, such as:

```
#!/usr/bin/python
```

After setting executable permissions, the script can be run directly using:

```
$chmod+xtest.py
$ ./test.py
```

This mode is widely used in software development because it provides better organization, scalability, and maintainability of code compared to interactive mode.

`

Overview of Python

Python is a high-level, interpreted, and dynamically typed programming language widely used in modern software development. It is well known for its simplicity, readability, and ease of learning, making it suitable for beginners as well as experienced developers. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming, which makes it highly flexible for developing a wide range of applications such as web development, data analysis, machine learning, and automation systems.**Features of Python**

· Simple and readable syntax

· Open-source and community-supported

· Cross-platform compatibility

· Extensive standard library and third-party packages

· Object-oriented programming (OOP) support

· Strong integration with other languages

Installing Python

Python can be downloaded from its official website https://www.python.org/ and installed on various operating systems including Windows, macOS, and Linux. During installation, users can also install package managers such as pip, which helps in installing additional libraries required for project development. Once installed, Python can be accessed through the command line or integrated development environments (IDEs) such as VS Code, PyCharm, or Jupyter Notebook.

Python Syntax and Basics

Python syntax is simple and easy to understand, which makes it ideal for rapid development. Programs are written using straightforward commands without the need for complex symbols or declarations. During installation, users can also install package managers such as pip, which helps in installing additional libraries required for project development.

`

Variables and Data Types

Python automatically assigns data types to variables based on the assigned value. Common data types include strings, integers, floating-point numbers, and boolean values. For example:

Name	:	"John"
age	:	30
height	:	5.9
is_student	:	False

Object-Oriented Programming (OOP) in Python

Python supports object-oriented programming (OOP), which is a programming paradigm based on the concept of classes and objects. OOP allows developers to structure programs in a modular and reusable way by combining data and functions within a single unit called a class.

A class acts as a blueprint for creating objects, while an object is an instance of a class. Python provides key OOP features such as encapsulation, inheritance, polymorphism, and abstraction, which help in building scalable and maintainable applications.

6.2.2 FLASK

Introduction to Flask

Flask is a lightweight and flexible web framework for Python that is widely used for developing web applications and APIs. It follows the WSGI (Web Server Gateway Interface) standard and provides essential tools required for web development without unnecessary complexity. Flask is highly preferred in machine learning and AI projects for deploying models as web applications due to its simplicity and scalability.

Features of Flask

Flask provides several important features that make it suitable for web development. It is lightweight and modular, allowing developers to add only the required components. It includes a built-in development server and debugger, which helps in testing applications efficiently. Flask uses the Jinja2 templating engine for dynamic HTML rendering and supports secure.

`

The cookie handling for session management. It also provides RESTful request handling and supports extensions for database integration, authentication, and other advanced functionalities.

Installing Flask

Flask can be installed easily using the Python package manager pip with the following command:

pip install flask

Creating a Flask Application

A simple Flask application can be created using the following code:

```
fromflaskimportFlask
app = Flask(__name__)

@app.route('/')
defhome():
return "Hello, Flask!"

if__name__=='__main__':
app.run(debug=True)
```

Running the Flask Application

The application can be executed using the command:

python app.py

After running, the application becomes accessible in the web browser at:

http://127.0.0.1:5000/

Flask Project Structure

A typical Flask project follows a structured format to organize files efficiently:

`

my_flask_app/
│──app.py
│──static/
│──templates/
│──config.py
│── requirements.txt

Here, app.py is the main application file, static/ contains CSS, JavaScript, and images, templates/ stores HTML files, config.py is used for configuration settings, and requirements.txt lists all required dependencies.

Flask Routing

Routing in Flask is used to define URL patterns and map them to specific functions in the application. It helps in handling user requests and returning appropriate responses. For example:

@app.route('/')
defhome():
return "Home Page"

6.2.3 HTML HTML (HyperText Markup Language):

HTML (HyperText Markup Language) is the standard markup language used to create and structure content on the World Wide Web. It forms the basic building block of web pages and defines the structure of content using a system of elements called tags. These tags are used to represent different parts of a webpage such as headings, paragraphs, images, links, tables, and forms.

HTML is not a programming language but a markup language, meaning it is used to structure and present content rather than perform logical operations. Every HTML document begins with a document type declaration, followed by the <html> tag, which encloses the entire webpage content.

Inside an HTML document, the structure is divided into two main sections: the <head> and the <body>. The <head> section contains metadata about the webpage, such as the title, character encoding, and links to external resources like CSS and JavaScript files. The <body> section contains all the visible content displayed to the user, including text, images, videos, and other multimedia elements.

HTML tags are usually written in pairs consisting of an opening tag and a closing tag, such as ... . However, some tags like <img> and
 are self-closing and do not require a closing tag. One of the most important features of HTML is the use of anchor tags <a> which allow navigation between different web pages through hyperlinks. Attributes within HTML tags provide additional information and control the behavior of elements, making webpages more interactive and structured.

6.3 MACHINE LEARNING & NLP

Machine Learning (ML) is a rapidly growing field of artificial intelligence that enables systems to learn patterns from data and make predictions or decisions without being explicitly programmed. The performance of machine learning models largely depends on the quality and quantity of data used for training. In ML projects, datasets are typically divided into training data and testing data.

The training dataset is the largest portion of the dataset and is used to train the machine learning model. During this phase, the model learns patterns and relationships from the input data. Once training is completed, the model is evaluated using the testing dataset, which contains unseen data. This helps to measure the model's performance and ability to generalize to new inputs. Typically, 70–80% of the data is used for training, while 20–30% is used for testing.

One of the commonly used algorithms in machine learning is the Random Forest algorithm. Random Forest is a supervised learning algorithm that can be used for both classification and regression tasks. It works by building multiple decision trees on different subsets of the dataset and combining their outputs to produce a final prediction. The final result is determined based on majority voting in classification problems or averaging in regression problems. This ensemble approach improves accuracy and reduces overfitting, making Random Forest a powerful and reliable model.

Model selection in machine learning refers to the process of choosing the most suitable algorithm for a given problem. It involves comparing different models based on their performance, complexity, and ability to generalize to unseen data.

Techniques such as cross-validation and grid search are commonly used to evaluate models. Performance metrics like accuracy, precision, recall, and mean squared error are used to identify the best-performing model that balances accuracy and efficiency.

Natural Language Processing (NLP) is a subfield of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language. It combines computational linguistics with machine learning techniques to process text and speech data. NLP is widely used in applications such as machine translation, speech recognition, chatbots, sentiment analysis, and text summarization.

With the increasing amount of digital text data from social media, websites, and communication platforms, NLP plays a crucial role in extracting meaningful insights and automating language-based tasks. In modern AI systems, NLP is often integrated with deep learning models to improve accuracy and contextual understanding of human language.

RANDOM FOREST ALGORITHM:

Random Forest is a widely used supervised machine learning algorithm that can be applied to both classification and regression problems. It is based on the concept of ensemble learning, where multiple models are combined to improve overall performance and accuracy. Instead of relying on a single decision tree, Random Forest builds a large number of decision trees using different subsets of the dataset.

Each decision tree in the Random Forest independently produces a prediction. For classification tasks, the final output is determined by majority voting among all trees, while for regression tasks, the final result is obtained by averaging the outputs of all trees. This collective decision-making process improves prediction accuracy and makes the model more robust.

One of the major advantages of Random Forest is its ability to reduce overfitting. As the number of trees increases, the model becomes more stable and generalizes better to unseen data. This makes Random Forest a reliable algorithm for handling complex datasets with high dimensionality.

MODEL SELECTION IN MACHINE LEARNING:

Model selection in machine learning refers to the process of identifying the most suitable algorithm and model configuration for a given dataset and problem the learning of the modles.

The perform differently depending on the nature of the data, selecting the right model is crucial for achieving optimal performance.

This process involves evaluating multiple algorithms and comparing their performance based on various criteria such as accuracy, error rate, computational efficiency, and ability to generalize to new data. Techniques like cross-validation and grid search are commonly used to systematically assess model performance and tune hyperparameters.

The main objective of model selection is to find a balance between model complexity and predictive performance. A well-selected model should not only perform well on training data but also generalize effectively to unseen data, ensuring reliable and consistent predictions in real-world applications.

Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language. It combines concepts from computer science, linguistics, and machine learning to process large amounts of textual and speech data.

NLP is widely used in various real-world applications such as machine translation, speech recognition, text summarization, sentiment analysis, and chatbot systems. It plays a key role in converting unstructured language data into meaningful and structured information that machines can analyze.

With the rapid growth of digital communication and social media platforms, the importance of NLP has increased significantly. It helps organizations and systems extract valuable insights from large volumes of text data and automate complex language-based tasks, thereby improving efficiency and decision-making capabilities.

NLP Techniques

Natural Language Processing (NLP) involves a wide range of techniques that enable computers to understand, interpret, and process human language effectively. These techniques are designed to convert unstructured text data into structured and meaningful information that can be used for analysis, prediction, and decision-making **NLP** techniques.

`

1. Text Processing and Preprocessing

Text preprocessing is the initial and most important step in NLP, where raw text data is cleaned and prepared for further analysis. This stage includes tokenization, which involves splitting text into smaller units such as words or sentences. Stemming and lemmatization are used to reduce words to their root form, while stopword removal eliminates commonly used words like “and”, “the”, and “is” that do not add significant meaning. Text normalization is also performed to standardize the text by converting it into a consistent format, such as lowercasing, removing punctuation, and correcting spelling errors.

2. Text Preprocessing

In addition to basic preprocessing, several detailed cleaning steps are applied to improve data quality. Tokenization divides text into meaningful units, while lowercasing ensures uniformity across the dataset. Stopword removal helps reduce noise by eliminating frequently occurring but less meaningful words. Punctuation removal further cleans the text by eliminating special characters. Stemming reduces words to their root form by cutting suffixes, whereas lemmatization converts words into their meaningful base form by considering context. Text normalization ensures consistency by handling spelling corrections, contractions, and special character removal.

3. Text Representation

- Bag of Words (BoW): Representing text as a collection of words, ignoring grammar and word order but keeping track of word frequency.
- Term Frequency-Inverse Document Frequency (TF-IDF): A statistic that reflects the importance of a word in a document relative to a collection of documents.
- Word Embeddings: Using dense vector representations of words where semantically similar words are closer together in the vector space (e.g., Word2Vec, GloVe).

4. Feature Extraction

Extracting meaningful features from the text data that can be used for various NLP tasks.

- N-grams : Capturing sequences of N words to preserve some context and word order.
- Syntactic Features: Using parts of speech tags, syntactic dependencies and parse trees.
- Semantic Features: Leveraging word embeddings and other representations to capture word meaning and context.

5. Model Selection and Training

Selecting and training a machine learning or deep learning model to perform specific NLP tasks.

- Supervised Learning: Using labeled data to train models like Support Vector Machines (SVM), Random Forests or deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs).
- Unsupervised Learning: Applying techniques like clustering or topic modeling (e.g., Latent Dirichlet Allocation) on unlabeled data.
- Pre-trained Models: Utilizing pre-trained language models such as BERT, GPT or transformer-based models that have been trained on large corpora.

6. Model Deployment and Inference

Deploying the trained model and using it to make predictions or extract insights from new text data.

- Text Classification: Categorizing text into predefined classes (e.g., spam detection, sentiment analysis).
- Named Entity Recognition (NER): Identifying and classifying entities in the text.
- Machine Translation: Translating text from one language to another.
- Question Answering: Providing answers to questions based on the context provided by text data.

7. Evaluation and Optimization

Evaluating the performance of the NLP algorithm using metrics such as accuracy, precision, recall, F1-score and others.

- Hyperparameter Tuning: Adjusting model parameters to improve performance.
- Error Analysis: Analyzing errors to understand model weaknesses and improve robustness.
- Text Classification: Categorizing text into predefined classes (e.g., spam detection, sentiment analysis).
- Named Entity Recognition (NER): Identifying and classifying entities in the text.
- Pre-trained Models: Utilizing pre-trained language models such as BERT, GPT or transformer-based models that have been trained on large corpora.

Technologies related to Natural Language Processing

There are a variety of technologies related to natural language processing (NLP) that are used to analyze and understand human language. Some of the most common include:

1. Machine learning: NLP relies heavily on machine learning techniques such as supervised and unsupervised learning, deep learning and reinforcement learning to train models to understand and generate human language.
2. Natural Language Toolkits (NLTK) and other libraries: NLTK is a popular open-source library in Python that provides tools for NLP tasks such as tokenization, stemming and part-of-speech tagging. Other popular libraries include spaCy, OpenNLP and CoreNLP.
3. Parsers: Parsers are used to analyze the syntactic structure of sentences, such as dependency parsing and constituency parsing.
4. Text-to-Speech (TTS) and Speech-to-Text (STT) systems: TTS systems convert written text into spoken words, while STT systems convert spoken words into written text.
5. Named Entity Recognition (NER) systems: NER systems identify and extract named entities such as people, places and organizations from the text.
6. Sentiment Analysis: A technique to understand the emotions or opinions expressed in a piece of text, by using various techniques like Lexicon-Based, Machine Learning-Based and Deep Learning-based methods
7. Machine Translation: NLP is used for language translation from one language to another through a computer.
8. Chatbots: NLP is used for chatbots that communicate with other chatbots or humans through auditory or textual methods.
9. AI Software: NLP is used in question-answering software for knowledge representation, analytical reasoning as well as information retrieval.
10. Sentiment Analysis: A technique to understand the emotions or opinions expressed in a piece of text, by using various techniques like Lexicon-Based, Machine Learning-Based and Deep Learning-based methods
11. Text-to-Speech (TTS) and Speech-to-Text (STT) systems: TTS systems convert written text into spoken words, while STT systems convert spoken words into written text.
12. Deep learning: NLP relies heavily on machine learning techniques such as supervised and unsupervised learning.

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTING

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined. System Test System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. Itis used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works. Unit Testing Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

• All field entries must work properly.

• Pages must be activated from the identified link.

• The entry screen, messages and responses must not be delayed.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results:

All the test cases mentioned above passed successfully. No defects encountered. That can been it is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works. Unit Testing Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined. System Test System testing ensures that the entire integrated software system meets requirements. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components. Unit Testing Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

7.2 Test Cases:

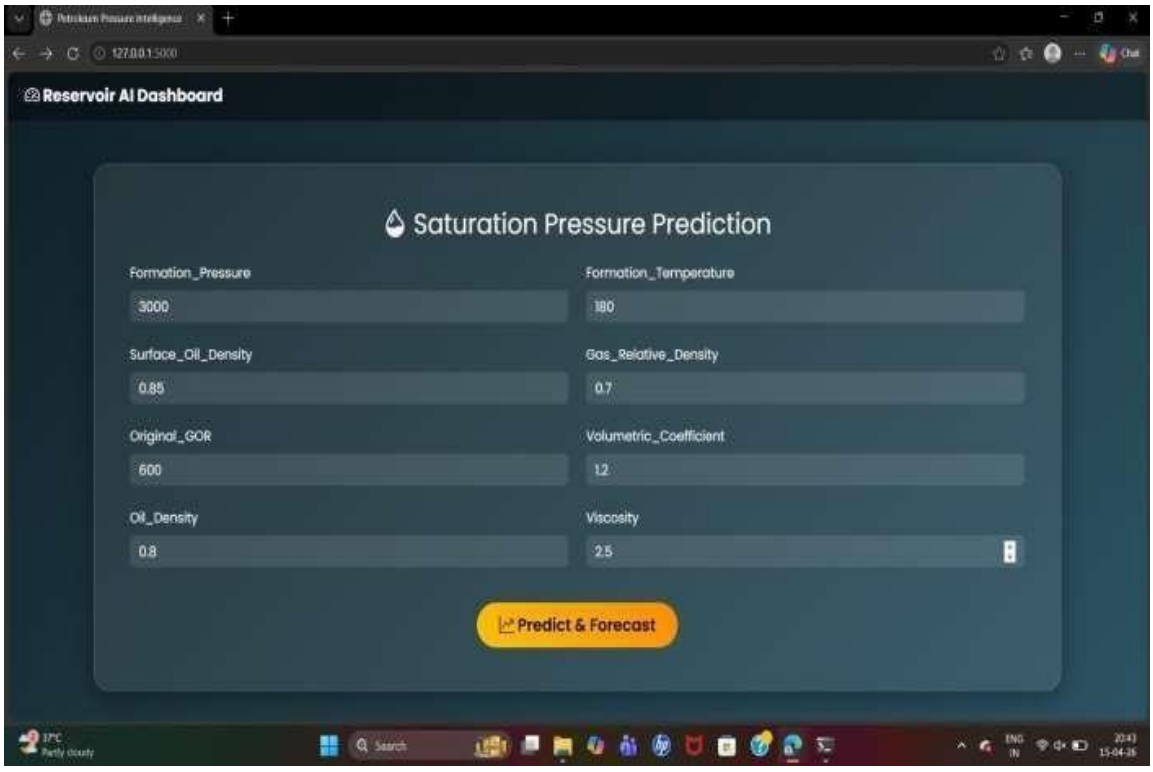
Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC_01	Upload valid speech input	Clear English speech audio file	Speech converted to text successfully	As expected	Pass
TC_02	Upload noisy speech input	Audio with background noise	Transcription with acceptable accuracy using Whisper	Slight noise distortion but readable text	Pass
TC_03	Upload text input	Simple English sentence	NLP processes text and extracts features correctly	As expected	Pass
TC_04	Speech feature extraction	Audio file input	MFCC, pitch, energy extracted using Librosa	As expected	Pass
TC_05	Text feature extraction	Paragraph input	Tokenization and POS tagging performed correctly	As expected	Pass
TC_06	Multimodal feature fusion	Speech + text input	Features combined into unified vector	Minor alignment delay in fusion	Pass
TC_07	Model prediction accuracy	Test dataset input	Stacking classifier predicts ~85% accuracy	83–85% accuracy variation observed	Pass
TC_08	Recommendation generation	Low grammar score input	Grammar improvement suggestions generated	Suggestions generated but not fully personalized	Pass
TC_09	Handle invalid input	Empty or unsupported file	System shows error/validation message	No proper error message in some cases	Fail

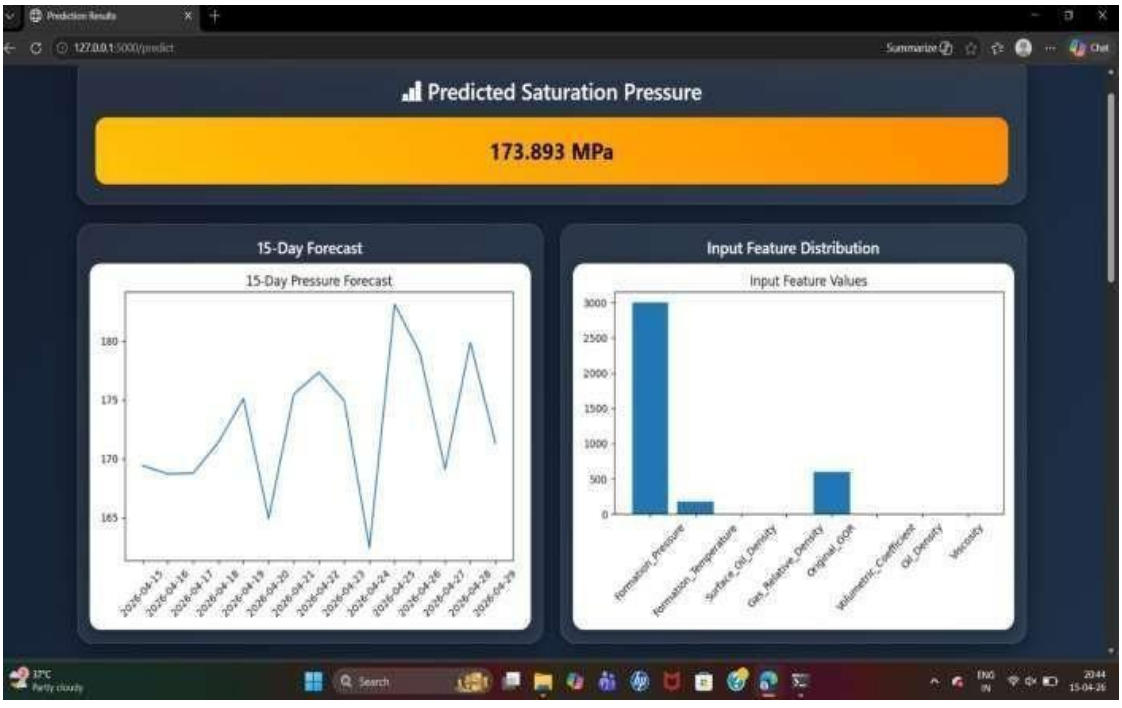
`

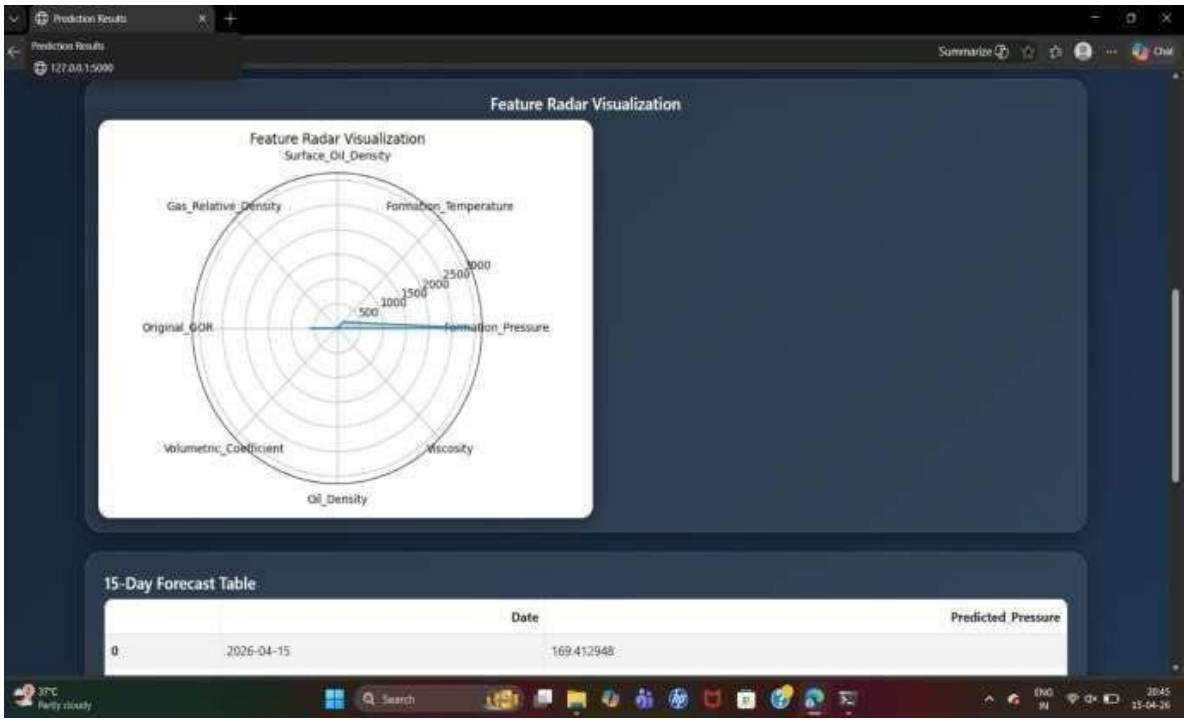
Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC_10	Real-time response	Large audio file upload	Fast processing and instant output	Delay observed for large files (>5MB)	Fail
TC_11	Model loading	First system run	All ML models load successfully	Whisper loading takes 20–30 sec	Fail
TC_12	Extreme noise input	Very low-quality audio	Partial transcription with errors	Highly inaccurate output	Fail

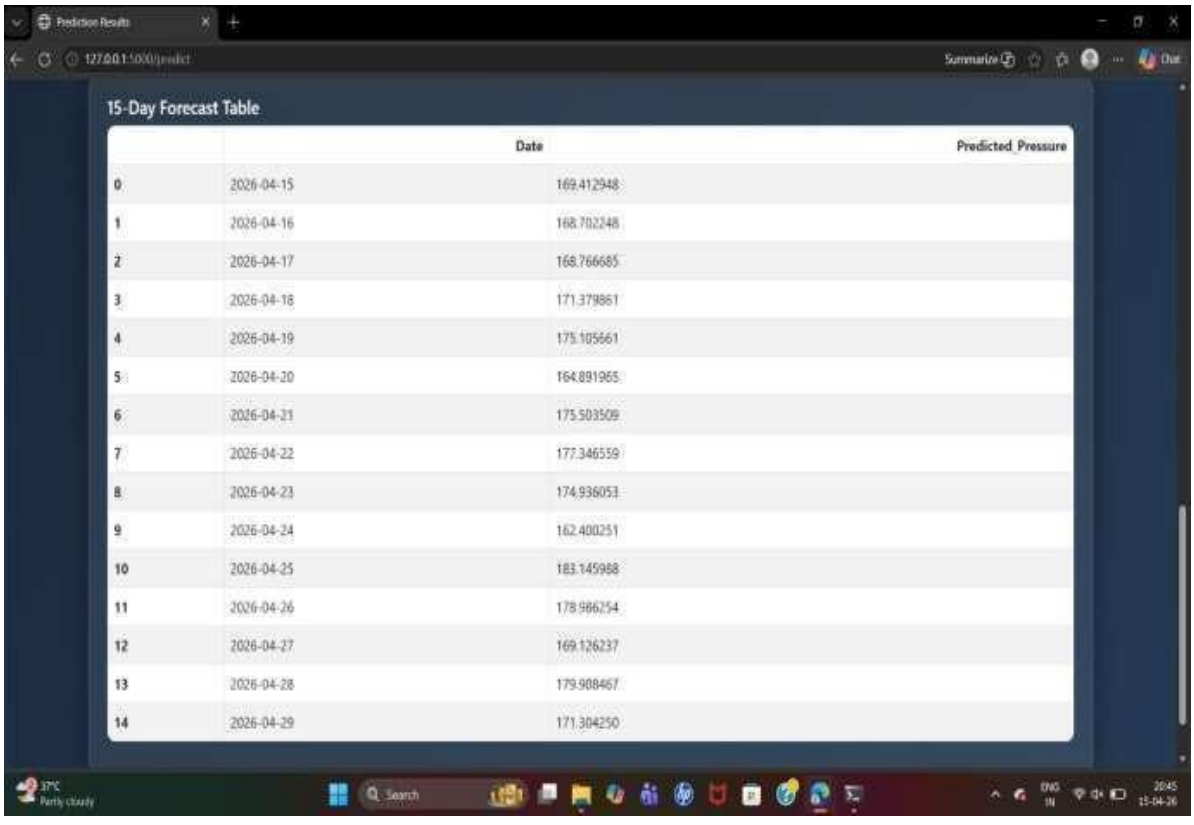
Table: 7.2 Test cases

8. OUTPUT SCREENS






Prediction Results
127.0.0.1:5000
Summarize
Feature Radar Visualization
Feature Radar Visualization
Surface_Oil_Density
Gas_Relative_Density
Formation_Temperature
500
1000
1500
2000
2500
3000
Original_GOR
Formation_Pressure
Volumetric_Coefficient
Viscosity
Oil_Density
15-Day Forecast Table
Date
Predicted_Pressure
0
2026-04-15
169.412948


Prediction Results
127.0.0.1:5000/predict
Summarize
15-Day Forecast Table
Date
Predicted_Pressure
0 2026-04-15 169.412948
1 2026-04-16 168.702248
2 2026-04-17 168.766685
3 2026-04-18 171.379861
4 2026-04-19 175.105661
5 2026-04-20 164.891965
6 2026-04-21 175.503509
7 2026-04-22 177.346559
8 2026-04-23 174.936053
9 2026-04-24 162.400251
10 2026-04-25 183.145988
11 2026-04-26 178.986254
12 2026-04-27 169.126237
13 2026-04-28 179.988467
14 2026-04-29 171.304250

9. CONCLUSION

This research presents an advanced AI-powered adaptive English language learning system that integrates multimodal data processing and ensemble machine learning techniques to enhance personalized learning experiences. The system effectively combines speech and text inputs using technologies such as Whisper for speech-to-text conversion, Librosa for audio feature extraction, and NLP techniques for textual analysis.

The core contribution of the proposed system lies in the implementation of a stacking ensemble classifier comprising Support Vector Machine (SVM), Random Forest, Decision Tree, and Gradient Boosting, along with a meta-classifier for optimized prediction. This ensemble approach significantly improves overall system performance and achieves an accuracy of approximately 85%, which is higher compared to traditional existing systems.

The system demonstrates strong capabilities in analyzing pronunciation, grammar, fluency, and vocabulary, while also providing real-time personalized feedback and learning recommendations. The integration of a Flask-based web application ensures an interactive user interface, ease of access, and scalability of the system.

Overall, the proposed system successfully addresses the limitations of traditional and existing AI-based learning models by providing a robust, efficient, and adaptive solution for modern English language learning and proficiency evaluation.

The core contribution of the proposed system lies in the implementation of a stacking ensemble classifier comprising Support Vector Machine (SVM), Random Forest, Decision Tree, and Gradient Boosting, along with a meta-classifier for optimized prediction.

The system demonstrates strong capabilities in analyzing pronunciation, grammar, fluency, and vocabulary, while also providing real-time personalized feedback and learning recommendations. The integration of a Flask-based web application ensures an interactive user interface, ease of access, and scalability of the system.

Unit Testing Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level.

10. FUTURE ENHANCEMENTS

The proposed AI-powered adaptive English language learning system can be further improved in several ways to enhance its performance, scalability, and usability. In the future, the system can be extended by integrating more advanced deep learning and transformer-based models such as GPT and improved versions of BERT to achieve better understanding of linguistic context and improve prediction accuracy beyond the current level.

Another major enhancement can be the inclusion of real-time conversational AI, where users can interact with the system in a dialogue-based environment. This would help learners practice spoken English more effectively in a natural and interactive manner.

The system can also be expanded by incorporating multilingual support, allowing users from different language backgrounds to learn English more efficiently through translation and code-mixed language processing.

In addition, the dataset size can be increased by collecting real-world speech samples from diverse accents, age groups, and proficiency levels. This will improve model generalization and robustness.

Future versions of the system can also include advanced emotion and sentiment detection to analyze the speaker's confidence, hesitation, and emotional tone during speech, providing deeper insights into learning behavior.

Furthermore, deployment can be enhanced using cloud platforms such as AWS or Azure to ensure better scalability, faster processing, and global accessibility. Mobile application support (Android/iOS) can also be developed to make the system more accessible and user-friendly.

Overall, these enhancements will make the system more intelligent, interactive, and widely applicable in real-world educational environments.

In the future, the system can be extended by integrating more advanced deep learning and transformer-based models such as GPT and improved versions of BERT to achieve better understanding of linguistic context and improve prediction accuracy beyond the current level. This will improve model generalization and robustness.

11. REFERENCES

1. M. Talebkeikhah, M. Nait Amar, A. Naseri, M. Humand, A. Hemmati-Sarapardeh, B. Dabir, and M. E. A. Ben Seghier, "Experimental measurement and compositional modeling of crude oil viscosity at reservoir conditions," *J. Taiwan Inst. Chem. Eng.*, vol. 109, pp. 35–50, Apr.2020.
https://doi.org/10.1016/j.jtice.2020.03.001
2. W. Dang et al., "Pore-scale mechanisms and characterization of light oil storage in shale nanopores,"*Geosci.Frontiers*,vol.13,no.5,2022.
https://doi.org/10.1016/j.gsf.2022.101424
3. Q. Wei et al., "Microscopic pore structure characteristics and fluid mobility in tight reservoirs,"*Minerals*,vol.13,no.8,p.1063,2023.
https://doi.org/10.3390/min13081063
4. A. K. Sleiti et al., "Comprehensive assessment and evaluation of correlations for gas-oil ratio…,"*J.PetroleumSci.Eng.*,vol.207,2021.
https://doi.org/10.1016/j.petrol.2021.109135
5. M. Wang et al., "Inflow performance analysis of a horizontal well…," *Lithosphere*, 2021.
https://doi.org/10.2113/2021/7024023
6. H. Nie et al., "Evaluation of gas content in organic-rich shale," *Geosci. Frontiers*, vol. 15, no.6,2024.
https://doi.org/10.1016/j.gsf.2024.101921
7. Y. Yang et al., "Evaluation and analysis of oil PVT correlation for Bohai heavy oilfield," *PetrochemicalIndAppl.*,2016.
https://doi.org/10.3969/j.issn.1673-5285.2016.06.004
8. J. Li et al., "Origin of abnormal pressure in the Upper Paleozoic shale…," *Mar. Petroleum Geol.*,vol.110,2019.
https://doi.org/10.1016/j.marpetgeo.2019.07.016
9. S. A. Kalogirou, "Artificial intelligence for modeling and control of combustion processes," *Prog. Energy Combust. Sci.*, 2003.
https://doi.org/10.1016/S0360-1285(03)00058-3
10. A. Mellit et al., "Artificial intelligence techniques for sizing photovoltaic systems," *Renew. Sustain.EnergyRev.*,2009.
https://doi.org/10.1016/j.rser.2008.01.006

11. J. Kennedy and R. Eberhart, "Particle swarm optimization," *Proc. ICNN*, 1995. https://doi.org/10.1109/ICNN.1995.488968
12. B. Liu et al., "Multiparameter logging evaluation of Chang 73 shale oil," *Geofluids*, 2023. https://doi.org/10.1155/2023/1672207
13. J. Sun et al., "Prediction of TOC content in organic-rich shale using ML algorithms," *Energies*,2023. https://doi.org/10.3390/en16104159
14. S. Akbar et al., "Deepstacked-AVPs: Predicting antiviral peptides…," *BMC Bioinformatics*,2024. https://doi.org/10.1186/s12859-024-05726-5
15. G. Rukh et al., "StackedEnCAOP: Prediction of antioxidant proteins…," *BMC Bioinformatics*,2024. https://doi.org/10.1186/s12859-024-05884-6
16. M. Ullah et al., "DeepAVP-TPPred: Identification of antiviral peptides…," *Bioinformatics*,2024. https://doi.org/10.1093/bioinformatics/btae305
17. J. Yang, "The high-pressure physical property characteristics of black oil reservoirs," China Univ. Petroleum, 2017. https://doi.org/10.1016/j.gsf.2024.101921
18. A. K. Sleiti et al., "Comprehensive assessment of gas-oil ratio correlations," *J. Petroleum Sci.Eng.*,2021. https://doi.org/10.1016/j.petrol.2021.109135
19. M. A. Al-Marhoun et al., "Prediction of crude oil viscosity curve using AI techniques," *J. PetroleumSciEng.*,2012. https://doi.org/10.1016/j.petrol.2012.03.029
20. A. K. Sleiti et al., "Comprehensive assessment and evaluation of correlations for gas-oil ratio…,"*J.PetroleumSci.Eng.*,vol.207,2011. https://doi.org/10.1016/j.petrol.2021.109135
21. M. Wang et al., "Inflow performance analysis of a horizontal well…," *Lithosphere*, 2011. https://doi.org/10.2113/2021/7024023